Problem 1 (6 pts)

Computational Problem. Find the minima of the following function by the Steepest Descent Method:

$$g(x_1, x_2) = \cos(x_1 + x_2) + \sin x_1 + \cos x_2$$

Let your code print the (x_1^2, x_2^n)'s generated by the Steepest Descent Method and the final minima (x_1^*, x_2^*), until a stopping criteria is reached. Take a screenshot of the results printed by your code, submit BOTH your code and the screenshot of the computation results.

For the steepest descent algorithm I let $\boldsymbol{x_0} = (0, 1)$. We let the code run with a tolerance of 10^{-6} and get a minima of $\boldsymbol{x}^* = (-1.04719695303.665191083)$.

```
>
> ## Run Steepest Descent x_0 = (0,1).
> print(SDM(g, 0, 1), digits = 10)
                 x          y
 [1,]  0.0000000000 1.000000000
 [2,] -0.2143613441 3.275657250
 [3,] -0.8007766579 3.240708523
 [4,] -0.8320489878 3.578067177
 [5,] -0.9821562597 3.558571122
 [6,] -0.9951591458 3.644906257
 [7,] -1.0324872486 3.639690073
 [8,] -1.0347047015 3.660463916
 [9,] -1.0434911212 3.659139787
[10,] -1.0442991727 3.664138428
[11,] -1.0463400946 3.663797132
[12,] -1.0465335978 3.664963399
[13,] -1.0469942845 3.664876300
[14,] -1.0470522526 3.665147136
[15,] -1.0471516548 3.665124247
[16,] -1.0471682499 3.665183903
[17,] -1.0471888509 3.665178156
[18,] -1.0471915212 3.665189703
[19,] -1.0471956167 3.665188681
[20,] -1.0471963999 3.665191169
[21,] -1.0471971977 3.665190923
>
```

Figure 1: Steepest Descent Algorithm Results

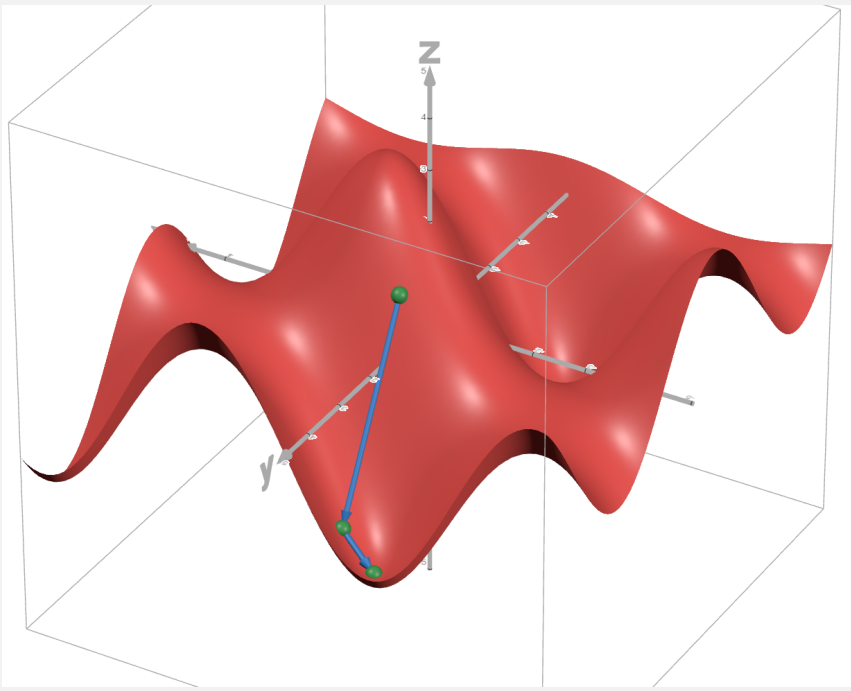

Figure 2: Trajectory of Steepest Descent Algorithm

I also plotted the function $g(x, y)$ as well as the points x_0, x_1, and x^* to see the trajectory that SDM takes in, seen in fig. 2. For code see (code appendix).

Problem 1 (6 pts)

Computational Problem. Solve the following system by Newton's method:

$$\begin{cases} x^2 + y^2 - 25 = 0 \\ x^2 - y - 2 = 0. \end{cases}$$

Let your code print the (x_n, y_n)'s generated by the Newton's method, until a stopping criteria is reached. Take a screenshot of the results printed by your code, submit BOTH your code and the screenshot of the computation results.

For my code I chose a tolerance of 10^{-8} and an initial value of $\boldsymbol{x_0} = (1, 4)$. Running the code we get,

```
> init <- c(1,4)
>
> NM(v,f,init,10,1e-8)
                x        y
results 1.000000 4.000000
        3.666667 4.333333
        2.695402 4.321839
        2.520406 4.321825
        2.514331 4.321825
        2.514324 4.321825
        2.514324 4.321825
>
```

Figure 3: Newton Method's Code Output

Let $x^* = (2.51432404047, 4.32182538050)$ we note that $|F(x^*)| < 7.9 \times 10^{-11}$. For code see (code appendix).

Problem 1 (8 pts)

Let $\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$. Consider the following system of linear equations $A\boldsymbol{x} = \boldsymbol{b}$,

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 = 0 \\ x_1 + 3x_2 + 4x_3 + 5x_4 = 1 \\ x_1 + 3x_2 + 5x_3 + 6x_4 = 1 \\ x_1 + 3x_2 + 5x_3 + 7x_4 = 1 \end{cases}.$$

(a) Find an LU factorization of A.

(b) Try to solve the system of linear equations using LU factorization.

(a) We write the system $Ax = b$ as

$$Ax = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 3 & 4 & 5 \\ 1 & 3 & 5 & 6 \\ 1 & 3 & 5 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} = b$$

We note that to get A into REF (the requirement for LU factorization) we need the following

elementary row operations, in order,

(1) $r_4 \leftarrow r_4 - r_3$, (2) $r_3 \leftarrow r_3 - r_2$, (3) $r_2 \leftarrow r_2 - r_1$.

Following these elementary row operations yields the system,

$$\left[\begin{array}{cccc|c} 1 & 2 & 3 & 4 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{array}\right] := Ux = c \tag{*}$$

Obviously we get our upper triangular matrix here however to get the lower triangular matrix let us write out the corresponding elementary matrices for the elementary row operations.

$$r_4 \leftarrow r_4 - r_3 : \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix} := L_1$$

$$r_3 \leftarrow r_3 - r_2 : \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} := L_2$$

$$r_2 \leftarrow r_2 - r_1 : \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} := L_3$$

We apply these in order to A and equate to the upper triangular matrix from $(*)$.

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{L_3} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{L_2} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix}}_{L_1} \underbrace{\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 3 & 4 & 5 \\ 1 & 3 & 5 & 6 \\ 1 & 3 & 5 & 7 \end{bmatrix}}_{A} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = U \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

Now we invert each elementary matrix to get

$$
\begin{aligned}
Ax &= L_1^{-1}L_2^{-1}L_3^{-1}Ux \\
Ax &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ +1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & +1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & +1 & 1 \end{bmatrix} Ux \\
Ax &= \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{U} x = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} = b
\end{aligned}
$$

Notably we have the LU factorization

$$
A = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{U}.
$$

(b) From $(*)$ we have the system

$$
\left[\begin{array}{cccc|c} 1 & 2 & 3 & 4 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{array}\right] = Ux = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}
$$

$$
\implies x = \begin{bmatrix} -2 \\ 1 \\ 0 \\ 0 \end{bmatrix}
$$

This is the solution.

Code Appendix

Algorithms are written in R.

Steepest Descent Algorithm

```r
## Steepest Descent Algorithm ##
library(rlang)
g <- expression(cos(x+y)+sin(x)+cos(y))

## Gradient at a point function (inputs: expression, output: eval at (x,y))

Grad <- function(f, x, y){
  grad_f <- Deriv(f, c("x", "y"))
  x <- x
  y <- y
  return(eval(grad_f))
}

## 3D Steepest Descent Algorithm (2D expression input)

SDM <- function(f, x_init, y_init, max_steps=100, precision=100){

  x_old <- x_init
  y_old <- y_init


  results <- matrix(c(x_old, y_old), ncol = 2, byrow = TRUE)

  #main loop
  i <- 1
  while (i<= max_steps){

    grad <- Grad(f, x_old, y_old)

    ## stopping criteria
    if (sqrt(sum(grad^2)) < 1e-6) break

    #create random positive scaling
    a_list <- rweibull(precision, 1.5, 1)

    #eval along descent line
    x_test <- x_old - a_list * grad[1]
    y_test <- y_old - a_list * grad[2]


    vals <- eval(f, env(x = x_test, y = y_test))

    ## pick minimizing step size
    a <- a_list[which.min(vals)]

    ## update
    x_new <- x_old - a * grad[1]
    y_new <- y_old - a * grad[2]

```

```r
    results <- rbind(results, c(x_new, y_new))

    x_old <- x_new
    y_old <- y_new

    i <- i + 1
  }

  colnames(results) <- c("x","y")
  results
}

## Run Steepest Descent x_0 = (0,1).
print(SDM(g, 0, 1), digits = 10)
```

Newton's Method Algorithm

```r
## Higher Dimensional Newton's Method ##

NM <- function(var_list, f_list, x_init, max_step, tol = 1e-6) {
  ## list of gradients (Jacobian rows)
  grad_list <- lapply(f_list, Deriv, x = var_list)

  ## initialize an empty matrix
  M <- matrix(NA_real_, nrow = length(f_list), ncol = length(var_list))

  ## starting point
  x_old <- x_init
  step <- 1

  ## store iterates
  results <- c(x_old)

  ## main loop
  while (step <= max_step){

    ## Inverse Jacobian at a point
    for (i in 1:length(var_list)){
      M[i,] <- eval(grad_list[[i]], list(x=x_old[1], y=x_old[2]))
    }
    inv_M <- solve(M)

    ## F eval at x_n
    f_x <- eval(f_list[1], list(x=x_old[1], y=x_old[2]))
    f_y <- eval(f_list[2], list(x=x_old[1], y=x_old[2]))
    f <- as.matrix(c(f_x, f_y), nrow = 2)

    ## NM update
    x_new <- x_old - inv_M %*% f

    ##record results
    results <- rbind(results, t(x_new))

    diff <- norm(x_new - x_old, type="2")
    if (diff < tol){
```

```
      break
    }

    ##loop back process
    x_old <- x_new
    step <- step + 1
  }

  colnames(results) <- var_list
  return(results)
}

## list of independent variables
v <- c("x","y")
## list of f_{i} for F: |R^{n} \to |R^{n}
f <- c(expression(x^2 + y^2 -25), expression(x^2 -y -2))
## x_{0}
init <- c(1,4)


## output results
print(NM(v,f,init,10,1e-8), digits =12)
```